

# DS Notebook Agent — Presentation & Report Prep

**One line:** a self-hostable **QLoRA fine-tuned Qwen3-4B**, wrapped in a LangGraph **Plan → Generate → Execute → Debug** loop with Pydantic-enforced cell outputs, that co-authors Jupyter notebook code from natural language — grounded in the live notebook state (DataFrame schemas, prior cell outputs). Research question: *can a small, fine-tuned, locally-served model inside a well-engineered agent loop approach the performance of much larger proprietary models, while keeping data private?*

**Rubric map:** Demo time §6 · Demo professionalism §6.2 · Problem/novelty §1 · Related work §2 · Evaluation measures §4 · Outcomes/analysis §5 · Report presentation (structure + figures §5.1, refs §7).

## 1. Problem & motivation (novelty)

LLMs are strong on clean coding benchmarks but weak on **realistic data science** code. On DS-1000 (1,000 StackOverflow-sourced DS problems across 7 libraries), even strong proprietary models struggle — GPT-4o-mini reaches only **42.2% accuracy** [Lai et al., 2023; see §5.1]. We attribute the gap to three failure modes a chat LLM cannot fix on its own:

1. **No awareness of live notebook state** — it doesn't know which DataFrames are loaded or their real columns/dtypes, so it guesses names.
2. **Manual error round-tripping** — the user must copy-paste tracebacks back in.
3. **Loose output** — chat prose / markdown instead of a strictly-formatted, immediately executable cell.

**Our thesis & scope.** We QLoRA fine-tune **Qwen/Qwen3-4B** (Apache-2.0, self-hostable) to close raw coding ability, and wrap it in a LangGraph agentic loop that directly attacks the three failure modes: it ingests notebook state, runs code in a sandbox and self-debuggs on real tracebacks, and enforces valid-cell output via Pydantic. We then benchmark **base vs. fine-tuned** checkpoints in the *same* pipeline on four DS suites. The system complements AutoML (Auto-sklearn, AutoGluon, H2O) — those tune models; we target the open-ended, exploratory notebook workflow. **Out of scope:** a full GUI. We focus on pipeline wiring (LangGraph), structured outputs, QLoRA fine-tuning, and comparative benchmarking.

## 2. Approach & agentic design

**Shape:** an applied LLM pipeline (context management, tool use, structured output) plus a targeted fine-tune. **Serving:** vLLM exposes an OpenAI-compatible endpoint; LangGraph is the client holding session state.

### 2.1 The graph

```
query → build_context → plan → generate → execute(sandbox) → success? -yes→
return
```



| no (≤3

```
retries)
```

```
└── debug ←┘
```

1. Take the query → 2. ingest notebook state (prior cells, stdout, DataFrame schemas) →
2. plan → 4. generate a cell → 5. execute in an isolated sandbox → 6. read stderr →
3. self-correct in a bounded loop, then return a structurally-valid cell.

## 2.2 Grounded in coding-agent practice (related work / differentiation)

| Node | Published pattern we build on | |---|---|---| | build\_context | **Agent-Computer Interface** [SWE-agent, Yang 2024] | | plan | **Plan-and-Solve** [Wang 2023] | | generate | **Act** step of **ReAct** [Yao 2022] | | execute | isolated action↔observation runtime [**OpenHands**, Wang 2024] | | debug | **Reflexion** [Shinn 2023] + **Self-Debug** [Chen 2023] |

**Differentiation vs. larger agents (DS-Star, Google 2025):** we deliberately keep a *single* small model with role-specialized *prompt profiles* rather than many large-model agents — cheaper to self-host while exercising the same plan→act→reflect contract.

## 2.3 Sandbox (safety)

Each cell runs in a disposable subprocess in our frozen DS venv (Python 3.10; pandas, numpy, scikit-learn, scipy, matplotlib, + **tensorflow, pytorch**) with a **60s timeout** and **2 GB RAM** cap; timeouts/OOM are parsed cleanly back into the debug loop.

## 3. Data & models

- **Training:** `jupyter-agent/jupyter-agent-dataset` (Apache-2.0, ~95.8k real-Kaggle triples of *notebook-context + query → verified cell + execution trace*). 10k sampled, 80/10/10 split; benchmark-overlapping samples filtered to prevent leakage.
- **Models:** base **Qwen3-4B** vs. **QLoRA fine-tune** (r=16, nf4, TRL), evaluated identically in the pipeline → isolates the fine-tuning contribution from pipeline effects.
- **Benchmarks:** DS-1000 (1,000), DABstep (450), plus a 50-task **leakage-safe dev/test** suite of realistic notebook scenarios grouped by analytic intent (§5.5). Published proprietary numbers (e.g., GPT-4o) are a *secondary, non-apples-to-apples* reference (not run through our pipeline).

## 4. Evaluation plan (appropriateness of measures)

**Why these measures fit code:** all grade by *executing* the code, not string similarity.

- **Pass@1 (execution success):** cell runs without raising.
- **Task accuracy:** passes the benchmark's hidden tests / matches the answer (DS-1000, DABstep accuracy).
- **Format compliance:** % of outputs that pass Pydantic validation as a valid, runnable cell.
- **Per-library breakdown + error taxonomy** localize *where* and *why* it fails.

**Controls:** same pipeline for base vs. fine-tuned (controlled comparison); `temperature=0` for plan/debug; bounded ≤3 retries (no infinite-loop reward-hacking); environment parity (TF/Torch installed

— otherwise those problems fail on `ModuleNotFound`).

**Ablation:** remove the review/debug loop → measure the drop in task accuracy (isolates the loop's value).

**Qualitative:** 100 held-out samples scored on (a) executability, (b) format compliance, (c) context awareness — reported as a/ab/ac/abc to attribute failures.

## 5. Results & analysis

Model: **Qwen3-4B-Instruct-2507**, evaluated as **base one-shot** vs. inside the **notebook agent** (same model, full plan→generate→execute→debug loop). Greedy decoding, TF+Torch in sandbox, bounded debug loop. Proprietary rows (GPT-\*) are published reference numbers, *not* run through our pipeline. *Note on the QLoRA fine-tune:* its objective is **multi-step reasoning, not raw code generation**, so we report it on the reasoning-oriented suites (DABStep, dev/test §5.5) rather than the pure code-generation benchmark DS-1000.

### 5.1 DS-1000 — full suite (1,000 problems), accuracy

| Name                                   | Model                      | Accuracy     |
|----------------------------------------|----------------------------|--------------|
| GPT-3.5-turbo                          | GPT-3.5-turbo              | 37.4%        |
| GPT-4o-mini                            | GPT-4o-mini                | 42.2%        |
| Qwen2.5-Coder-14B-Instruct             | Qwen2.5-Coder-14B-Instruct | 32.3%        |
| Qwen3-4B-Instruct-2507 (base one-shot) | Qwen3-4B-Instruct-2507     | 27.2%        |
| <b>Notebook agent</b>                  | Qwen3-4B-Instruct-2507     | <b>30.2%</b> |

**Headline:** wrapping the *same* 4B model in the agent loop lifts DS-1000 accuracy **27.2% → 30.2% (+3.0 pts)** — a 4B local model in our pipeline closes most of the gap to GPT-3.5-turbo (37.4%) and beats the 14B Qwen2.5-Coder (32.3%)... (*beats / trails — adjust once final*).

### 5.2 DS-1000 — per-library breakdown

Counts in parentheses are problems-passed. We report **base vs. notebook agent** here; the QLoRA fine-tune is deliberately omitted — DS-1000 is a pure **code-generation** suite, whereas the fine-tune targets **multi-step reasoning** (its contribution is reported on the reasoning suites in §5.4–§5.5).

| Library          | Notebook agent  | Base one-shot |
|------------------|-----------------|---------------|
| Pandas (291)     | 16% (47)        | 16% (46)      |
| NumPy (220)      | 38% (84)        | 35% (77)      |
| Matplotlib (155) | <b>48% (75)</b> | 48% (75)      |
| Sklearn (115)    | 25% (29)        | 19% (22)      |
| SciPy (106)      | 26% (28)        | 25% (27)      |
| PyTorch (68)     | <b>32% (22)</b> | 18% (12)      |

| Library         | Notebook agent          | Base one-shot   |
|-----------------|-------------------------|-----------------|
| TensorFlow (45) | <b>38% (17)</b>         | 28% (12)        |
| <b>Overall</b>  | <b>30.2% (302/1000)</b> | 27.2% (271/996) |

### 5.3 What it tells us

- **The agent loop helps the base model everywhere it matters:** notebook agent  $\geq$  base one-shot in every library — biggest jumps on PyTorch (18% $\rightarrow$ 32%) and TensorFlow (28% $\rightarrow$ 38%), where notebook-state grounding + environment parity supply exact API context.
- **Why no fine-tune column here:** DS-1000 measures single-shot *code generation*, not the multi-step *reasoning* the QLoRA fine-tune is trained for — so improving it is not the fine-tune's job. We isolate the fine-tune's contribution on the reasoning-oriented suites instead (DABStep §5.4, dev/test §5.5).
- **Pandas is the standing bottleneck** (~16% across configs): subtle dtype/shape/index logic that the scaffolding alone has not cracked.

### 5.4 DABStep (leaderboard submission)

**dabstep** profile; Easy / Hard accuracy. Same model, base vs. notebook agent.

| Name                          | Model                  | Easy         | Hard |
|-------------------------------|------------------------|--------------|------|
| Qwen3-4B-Instruct-2507 (base) | Qwen3-4B-Instruct-2507 | 44.0%        | 2.1% |
| <b>Notebook agent</b>         | Qwen3-4B-Instruct-2507 | <b>52.8%</b> | 2.1% |

- **Easy: 44.0%  $\rightarrow$  52.8% (+8.8 pts)** — the agent loop's largest single win; multi-step context + self-debug pays off on the tractable analytic tasks.
- **Hard: flat at 2.1%** — multi-hop reasoning over large schemas remains out of reach for a 4B model regardless of scaffolding; this is the headroom the fine-tune targets next.

### 5.5 Quality check

Our own **leakage-safe** suite of 30 realistic single-DataFrame notebook scenarios, grouped by analytic *intent* (each graded by executing the cell and checking the answer). This is the workload the system is actually built for — natural-language requests against live notebook state — and the set we use to tune the agent. Notebook agent, Qwen3-4B base.

| Intent            | Notebook agent  |
|-------------------|-----------------|
| Statistics (4)    | <b>100% (4)</b> |
| Visualization (7) | 100% (7)        |
| Preprocessing (3) | 100% (3)        |
| Exploration (7)   | 86% (6)         |
| Aggregation (6)   | 83% (5)         |
| Filtering (3)     | 67% (2)         |

| Intent  | Notebook agent |
|---------|----------------|
| Overall | 90% (27/30)    |

Format compliance **100%** (30/30 outputs pass Pydantic validation as runnable cells); all 3 misses are genuine wrong answers, not harness errors (0 errors in this run).

- **Intent matters more than library:** on realistic, well-scoped notebook asks the agent reaches **90%** overall — far above its DS-1000 (30%) and DABStep-Hard (2%) numbers — because these tasks supply concrete notebook state and a single clear goal.
- **Strong where output is checkable and local** (statistics, visualization, preprocessing all at 100%); **weakest on filtering** (67%) — precise boolean/index conditions remain the failure mode, mirroring the Pandas bottleneck in §5.3.

5.6 Leakage audit — benchmarks vs. training corpus

We verified the headline benchmarks are **not contaminated** by the training data. We scanned every DS-1000 and DABStep unit against all **102,778** training documents from `jupyter-agent/jupyter-agent-dataset` (both `non_thinking` and `thinking` splits) using **8-gram containment**, flagging any unit  $\geq 0.8$ .

| Group              | Units | Mean containment | Max containment | $\geq 0.5$ | $\geq 0.8$ |
|--------------------|-------|------------------|-----------------|------------|------------|
| DABStep / question | 454   | 0.0006           | 0.286           | 0          | 0          |
| DS-1000 / prompt   | 1,000 | 0.012            | 0.161           | 0          | 0          |
| DS-1000 / solution | 1,000 | 0.000            | 0.000           | 0          | 0          |
| Total              | 2,454 | —                | —               | 0          | 0          |

- **No leakage detected:** 0 of 2,454 units flagged. The single highest overlap anywhere was **0.286** (one DABStep question) — well below the 0.5 caution line and the 0.8 flag threshold.
- **DS-1000 solutions show exactly zero overlap** — the decisive signal, since solution leakage is what would inflate accuracy. Our §5.1–§5.4 numbers reflect genuine generalization, not memorization.

7. References

**Agent patterns:** ReAct (Yao 2022, arXiv:2210.03629) · Plan-and-Solve (Wang 2023, arXiv:2305.04091) · Reflexion (Shinn 2023, arXiv:2303.11366) · Teaching LLMs to Self-Debug (Chen 2023, arXiv:2304.05128) · SWE-agent (Yang 2024, arXiv:2405.15793) · OpenHands (Wang 2024, arXiv:2407.16741) · DS-Star (Google Research, 2025).

**Benchmarks:** DS-1000 (Lai 2023, arXiv:2211.11501) · DABStep (Cabannes 2025, arXiv:2506.23719).

**Model & data:** Qwen3 Technical Report (2025, arXiv:2505.09388) · `jupyter-agent/jupyter-agent-dataset` (HF, 2025).

**Source map:** graph `src/agent/graph.py` · nodes `src/agent/nodes.py` · prompt profiles `src/agent/prompts.py` · sandbox `src/agent/sandbox.py` · eval `eval/harness.py`,

eval/benchmarks/\*.py.